JOB CORE · REACT QUERY · PHASES 3–4

React Query — Lesson 02: Caching & Mutations

Caching, Sync, Refetch Triggers, useMutation, Invalidation & Server Updates

Source Lesson: Caching, Sync, Mutation & Server Update

HOW TO USE THIS · PHASES 3–4 ROADMAP

Master Core Cache and Mutation Patterns First

Study Vault Review Workflow

- **Checklist PDF Review:** Review each slide like a structured checklist before coding.
- **Core (Know These):** Master staleTime, gcTime, refetching, useMutation, and sync.
- **Mention (Learn Later):** Treat polling, skipToken, and concurrent rollbacks as deep dives.
- **Study Vault & Labs:** Toggle explanations in Study Vault and build Projects 2 & 3.



Control Freshness With staleTime and Memory Eviction With gcTime

How React Query Caches by Key (Core)

- **Four Cache States:** Queries transition across fresh, stale, active, and inactive.
- **Shared Cache:** Components using the same key share one cache entry (Mention).

Garbage Collection & Timing (Core + Mention)

- **Garbage Collection:** Inactive cache stays in memory until the gcTime timer expires.
- **Canonical Timeline:** Mount (fresh) -> stale -> unmount (inactive) -> GC deletes entry.

Cache Knob / State	Default & Trigger	What It Controls in Your App
staleTime (Freshness)	Default: 0 ms (data is stale immediately).	How long cached data stays fresh without background refetching.
gcTime (Memory GC)	Default: 5 minutes after last observer unmounts.	How long inactive cache remains in memory before garbage collection.
Active vs Inactive	Based on mounted component observers.	Active queries have >=1 mounted subscriber; inactive queries have 0.

10–12. REFETCH · RETRY · ENABLED (CORE)

Auto-Refetch on Focus and Gate With enabled

Refetch Triggers, Retries & Gating

- **Focus · Mount · Reconnect:** Stale queries auto-refetch on tab focus, mount, or network recovery.
- **Background + Manual:** Auto refetches sync silently; call refetch() for user button clicks.
- **Retry Defaults:** Retries failed requests 3 times; disable retry on 404 or 401 errors.
- **enabled & Dependent:** Use enabled: Boolean(userId) so dependent queries wait for inputs.



10–12. ADVANCED FETCH CONTROL · MENTION (LEARN LATER)

Extend Fetching With Polling, Backoff, and Auth Guards

Polling & Backoff

- **refetchInterval:** Poll live endpoints on a fixed schedule (such as every 5s).
- **Exponential Backoff:** Wait time doubles between retries so overloaded APIs recover.

skipToken Gating

- **skipToken Pattern:** Type-safe TanStack alternative to enabled: false in TypeScript.
- **Dependent Chains:** Pause child queries cleanly until parent IDs or params exist.

Auth-Gated Fetch

- **Session Readiness:** Hold protected queries until the user auth token is loaded.
- **Smart Error Rules:** Skip retries on 401/403 responses and redirect to login.

13–14. USEMUTATION · CORE + MENTION

Run Server Writes With useMutation Callbacks

Server Writes & Lifecycle Callbacks

- **Read vs Write (Core):** useQuery reads cached data; useMutation performs POST/PUT/DELETE.
- **mutate & mutateAsync:** Pass mutationFn; call mutate() for callbacks or mutateAsync() to await.
- **States & Callbacks:** Track pending/error/success and run onSuccess, onError, onSettled.
- **Mention Deep Dives:** onMutate pre-flight hook, reset() state cleanup, and API error shapes.



Sync Server Writes via Invalidation, Cache Writes, or Rollbacks

Core Sync Pattern	Mechanism & API	When to Use It
invalidateQueries	Marks matching key prefixes stale and refetches.	Default choice after mutations when server computes state.
setQueryData / getQueryData	Reads or writes cache directly in memory.	When mutation response already returns the updated record.
Optimistic Update	Snapshot -> update cache -> rollback on error.	Instant UI response for toggles, likes, and todo edits.

Invalidate vs Direct Cache Write

- **Partial Key Match:** Invalidating ['todos'] refreshes ['todos', { status: 'done' }].
- **Direct Cache Write:** setQueryData updates UI immediately without a network refetch.

Three-Step Optimistic Flow

- **1. Snapshot & Write:** Read prior state via getQueryData, then apply optimistic edit.
- **2. Rollback on Error:** Restore the saved snapshot in onError if the request fails.

15–17. ADVANCED CACHE UPDATES · MENTION (LEARN LATER)

Guard Optimistic Writes With Immutability

Deep-Dive Cache Update Mechanics

- **Cancel Outgoing Queries:** Call cancelQueries in onMutate so in-flight reads do not clobber UI.
- **Immutable Cache Updates:** Always return a new array or object in setQueryData, never mutate.
- **setQueriesData Bulk Edit:** Update all matching cache entries at once using a key prefix.
- **Concurrent & Full Pattern:** Handle overlapping mutations and resync server state in onSettled.



PRACTICE & REVIEW · FULL LESSON.MD

Build Projects 2 and 3 in Full lesson.md to Master Phases 3–4

Projects 2 & 3

- **Project 2: Explorer:** Build Product Explorer with staleTime, gcTime, and enabled.
- **Project 3: Todo App:** Implement mutations, invalidation, and optimistic rollbacks.

Self-Check Quiz

- **Cache & Refetch:** Contrast staleTime vs gcTime and trace focus/mount refetches.
- **Mutation & Sync:** Choose between invalidateQueries, setQueryData, and rollback.

Phase 5 Preview

- **What's Next: Phase 5:** Advance to pagination, infinite queries, and prefetching.
- **If You Missed Class:** Open full lesson.md in Study Vault and toggle Mention blocks.

LESSON 02 COMPLETE · STUDY VAULT

Tune Freshness With staleTime. Sync Writes With Mutations.

Open lesson.md in Study Vault to build Projects 2 and 3.